TRABAJO PRÁCTICO 3 – OBSERVER

Sistema de inventario que notifica a múltiples servicios

Metodología de Sistemas II – Tecnicatura Universitaria en Programación a Distancia – UTN

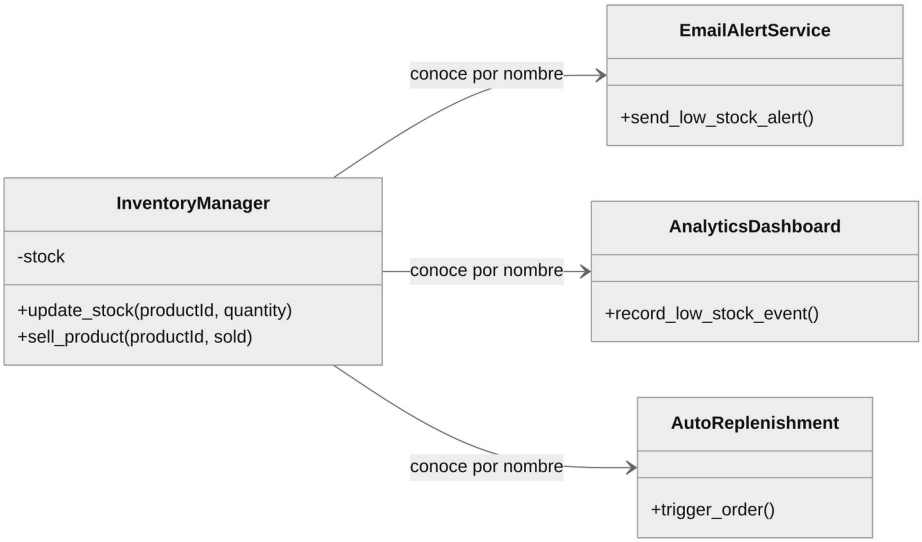
Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26
Fecha de entrega:	29/10/2026
Lenguaje:	Python 3 (sin librerías externas)

1. El problema

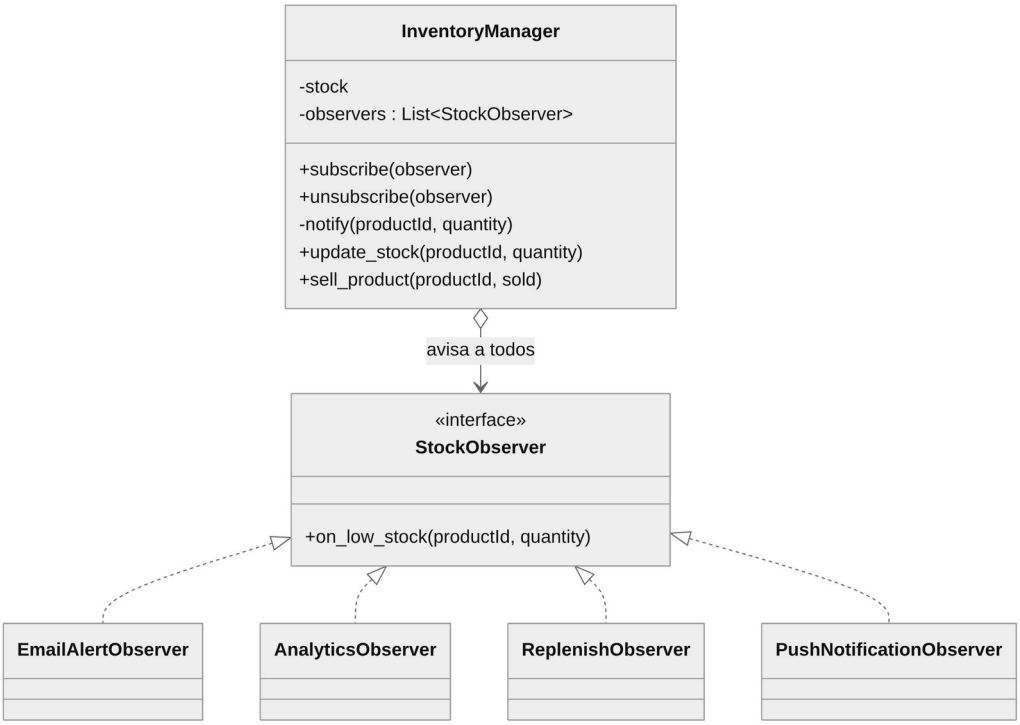
InventoryManager crea y llama directamente a EmailAlertService, AnalyticsDashboard y AutoReplenishment. Si se quiere agregar otra notificación, o si cambia alguno de esos servicios, hay que modificar el manager, que no tendría que saber nada de mails ni de analytics. Además, el mismo bloque de avisos está repetido en dos métodos, y si un servicio falla corta a los demás.

2. Diagramas UML

Antes



Después



El manager pasó de conocer tres clases concretas a conocer solo la interfaz StockObserver. Los observadores se suscriben desde afuera.

3. Código antes

antes.py

```
# ANTES: código original (problemático)
# InventoryManager conoce por nombre a los tres servicios.


class EmailAlertService:
    def send_low_stock_alert(self, product_id, quantity):
        print(f"  [Email] Stock bajo de {product_id}: quedan {quantity}")


class AnalyticsDashboard:
    def record_low_stock_event(self, product_id):
        print(f"  [Analytics] Evento de stock bajo registrado para {product_id}")


class AutoReplenishment:
    def trigger_order(self, product_id, amount):
        print(f"  [Reposición] Orden de {amount} unidades de {product_id}")


class InventoryManager:
    def __init__(self):
        self.stock = {}
        self.email_service = EmailAlertService()
        self.analytics = AnalyticsDashboard()
        self.replenishment = AutoReplenishment()
```

```
    def update_stock(self, product_id, quantity):
        self.stock[product_id] = quantity
        if quantity < 10:
            self.email_service.send_low_stock_alert(product_id, quantity)
            self.analytics.record_low_stock_event(product_id)
            self.replenishment.trigger_order(product_id, 100)

    def sell_product(self, product_id, sold):
        self.stock[product_id] -= sold
        if self.stock[product_id] < 10:
            # el mismo bloque repetido
            self.email_service.send_low_stock_alert(product_id, self.stock[product_id])
            self.analytics.record_low_stock_event(product_id)
            self.replenishment.trigger_order(product_id, 100)


if __name__ == "__main__":
    manager = InventoryManager()
    manager.update_stock("PROD-001", 5)
    # Para agregar notificaciones push habría que abrir InventoryManager.
    # Y si el email falla, la excepción corta todo y no se hace la reposición.
```

4. Código después

despues.py

```
# DESPUÉS: refactorizado con Observer
from abc import ABC, abstractmethod

LIMITE_STOCK_BAJO = 10


# --- Interfaz del observador ---
class StockObserver(ABC):
    @abstractmethod
    def on_low_stock(self, product_id, quantity): ...


# --- Observadores concretos ---
class EmailAlertObserver(StockObserver):
    def on_low_stock(self, product_id, quantity):
        print(f"  [Email] Stock bajo de {product_id}: quedan {quantity}")


class AnalyticsObserver(StockObserver):
    def on_low_stock(self, product_id, quantity):
        print(f"  [Analytics] Evento de stock bajo registrado para {product_id}")


class ReplenishObserver(StockObserver):
    def on_low_stock(self, product_id, quantity):
        print(f"  [Reposición] Orden de 100 unidades de {product_id}")


class PushNotificationObserver(StockObserver):
    # Observador nuevo: se agregó sin tocar InventoryManager.
```

```
    def on_low_stock(self, product_id, quantity):
        print(f"  [Push] Aviso al celular del encargado: {product_id} tiene {quantity} unidades")


# --- Notificador ---
class InventoryManager:
    def __init__(self):
        self.stock = {}
        self.observers = []        # no conoce a ningún observador por nombre

    def subscribe(self, observer: StockObserver):
        self.observers.append(observer)

    def unsubscribe(self, observer: StockObserver):
        self.observers.remove(observer)

    def _notify(self, product_id, quantity):
        for observer in self.observers:
            # El try va DENTRO del for: si un observador falla,
            # se registra el error y se sigue con el siguiente.
            try:
                observer.on_low_stock(product_id, quantity)
            except Exception as error:
                print(f"  [ERROR] {type(observer).__name__} falló: {error}")

    def update_stock(self, product_id, quantity):
        self.stock[product_id] = quantity
        if quantity < LIMITE_STOCK_BAJO:
            self._notify(product_id, quantity)

    def sell_product(self, product_id, sold):
        self.stock[product_id] -= sold
        if self.stock[product_id] < LIMITE_STOCK_BAJO:
            self._notify(product_id, self.stock[product_id])


# --- Prueba del observador que falla (obligatoria) ---
class ObservadorRoto(StockObserver):
    def on_low_stock(self, product_id, quantity):
        raise ConnectionError("error de red simulado")


if __name__ == "__main__":
    manager = InventoryManager()
    manager.subscribe(EmailAlertObserver())
    manager.subscribe(AnalyticsObserver())
    manager.subscribe(ObservadorRoto())
    manager.subscribe(ReplenishObserver())

    print("1) Stock bajo con un observador roto en el medio:")
    manager.update_stock("PROD-001", 5)

    print("\n2) Agrego un observador nuevo (push) sin tocar el manager:")
    manager.subscribe(PushNotificationObserver())
    manager.update_stock("PROD-002", 50)   # stock normal: no avisa
    manager.sell_product("PROD-002", 45)   # baja a 5: avisa a todos
```




```
    print("\n3) Desuscribo el observador roto:")
    roto = next(o for o in manager.observers if isinstance(o, ObservadorRoto))
    manager.unsubscribe(roto)
    manager.update_stock("PROD-003", 2)
```

5. Ejecución y test del observer roto

```
$ python despues.py
```

```
1) Stock bajo con un observador roto en el medio:
  [Email] Stock bajo de PROD-001: quedan 5
  [Analytics] Evento de stock bajo registrado para PROD-001
  [ERROR] ObservadorRoto falló: error de red simulado
  [Reposición] Orden de 100 unidades de PROD-001

2) Agrego un observador nuevo (push) sin tocar el manager:
  [Email] Stock bajo de PROD-002: quedan 5
  [Analytics] Evento de stock bajo registrado para PROD-002
  [ERROR] ObservadorRoto falló: error de red simulado
  [Reposición] Orden de 100 unidades de PROD-002
  [Push] Aviso al celular del encargado: PROD-002 tiene 5 unidades

3) Desuscribo el observador roto:
  [Email] Stock bajo de PROD-003: quedan 2
  [Analytics] Evento de stock bajo registrado para PROD-003
  [Reposición] Orden de 100 unidades de PROD-003
  [Push] Aviso al celular del encargado: PROD-003 tiene 2 unidades
```

En la prueba 1, ObservadorRoto lanza una excepción, pero se registra el error y la reposición, que está suscripta después, igual se ejecuta. Esto funciona porque el try/except está dentro del for: si estuviera afuera, el primer error cortaría el recorrido. En la prueba 2 se agrega PushNotificationObserver sin modificar InventoryManager, y también se ve que sell_product avisa a todos.

6. Justificación

El problema concreto era el acoplamiento: el manager de inventario tenía que conocer a cada servicio que quería enterarse del stock bajo. Con Observer, el manager solo tiene una lista de observadores y les avisa a todos llamando a on_low_stock(), sin saber quiénes son. Agregar las notificaciones push fue crear una clase y suscribirla, sin tocar el manager. También saqué el bloque repetido: update_stock y sell_product ahora llaman al mismo _notify(). Y al manejar los errores de cada observador por separado, una falla, por ejemplo de red en el mail, ya no impide que se haga el pedido de reposición.

7. Diferencias con la versión canónica

- La teoría usa un método update() genérico. Yo lo llamé on_low_stock() porque es el único evento que hay y el nombre dice mejor qué pasó. Si después hubiera más eventos (por ejemplo, stock repuesto), cambiaría a un método más general que reciba el tipo de evento.
- El umbral de stock bajo (10) quedó en una constante, LIMITE_STOCK_BAJO, para no repetir el número. Las notificaciones son síncronas y no usé threads, como pide la consigna.